

Implementation Status

Feature support matrix across reference implementations.

Quick Summary

Implementation	Decode	Encode	Binary Schema	Performance
Python	Full	Full	Full	45K msg/s
Go	Full	Partial	Full	2.1M msg/s
C	Full	-	Full	32M msg/s
JavaScript	Full	Partial	-	180K msg/s

Detailed Feature Matrix

Core Types

Feature	Python	Go	C	JS
u8, u16, u32, u64				
s8, s16, s32, s64				
u24, s24				
f16 (half-precision)				
f32, f64				
bool				
ascii				
hex				
bytes				
base64				
skip				
enum				
udec / sdec				
bitfield_string				-

Bitfield Syntax

Feature	Python	Go	C	JS
Range u8[0:3]				
Width u8:4				
Cross-byte u16[4:11]				
byte_group				
Endian prefix le_u16				

Arithmetic Modifiers

Feature	Python	Go	C	JS
add				
mult				
div				
YAML key ordering				
lookup (array)				

Feature	Python	Go	C	JS
lookup (map)				

Transform Pipeline

Feature	Python	Go	C	JS
sqrt				
abs				
pow				
log / log10				
floor / ceiling				
clamp				
round			-	

Computed Fields

Feature	Python	Go	C	JS
type: number				
ref: \$field				
polynomial				
compute: {op, a, b}			-	
guard conditions			-	

Conditional Parsing

Feature	Python	Go	C	JS
switch				
switch range 2..5			-	
switch default _				
flagged				
tlv				
match_value			-	

Structures

Feature	Python	Go	C	JS
type: object				
type: repeat (count)				
repeat (count_field)				
repeat (until: end)				
definitions / use			-	
ports (fPort routing)				
var (variables)			-	

Encodings

Feature	Python	Go	C	JS
<code>sign_magnitude</code>				
<code>bcd</code>				
<code>gray</code>		-	-	

Downlink Support (v0.3.2)

Feature	Python	Go	C	JS
<code>direction</code> property		-	-	
<code>downlink_commands</code>		-	-	
<code>encode_command()</code>		-	-	
<code>decode_command()</code>		-	-	
Bidirectional schemas		-	-	

Schema Composition (v0.3.2)

Feature	Python	Go	C	JS
<code>\$ref:</code> cross-file		-	-	-
<code>use:</code> shorthand		-	-	-
<code>rename:</code> fields		-	-	-
<code>prefix:</code> fields		-	-	-
Compact format strings		-	-	-

Validation (v0.3.2)

Feature	Python	Go	C	JS
ERROR level		-	-	-
WARNING level		-	-	-
INFO level		-	-	-
Best practice checks		-	-	-
Quality scoring		-	-	-

Semantic Hints

Feature	Python	Go	C	JS
<code>unit</code>		-		
<code>ipso</code>		-		
<code>senml_unit</code>		-		
<code>description</code>		-		

Binary Schema Format

Feature	Python	Go	C	JS
Parse v1			-	
Parse v2			-	

Feature	Python	Go	C	JS
Encode v1			-	-
Encode v2			-	-
QR encoding		-	-	-

Encoding (Struct→Binary)

Feature	Python	Go	C	JS
Basic types			-	
Bitfields		Partial	-	
Nested objects		-	-	
Repeat		-	-	
Conditionals		-	-	-

Implementation Notes

Python (`tools/schema_interpreter.py`)

Reference implementation - most complete and tested.

- Full decode and encode support
- All schema features implemented (v0.3.2 spec)
- Extensive test coverage (477+ tests)
- Binary schema encode/decode
- Used for validation and code generation
- **v0.3.2 additions:** `downlink_commands`, `direction`, `encodings`, `compact format`

Go (`go/schema/`)

Production quality for high-throughput servers.

- Full decode support
- YAML and binary schema parsing
- Optimized for performance (2.1M msg/s with binary)
- Missing: `definitions`, `guard`, some encodings
- Encode support partial (basic types only)

C (`include/schema_interpreter.h`)

Embedded-optimized - no dynamic allocation required.

- Full decode support
- Binary schema loading (no YAML)
- Programmatic schema building
- 32M msg/s throughput
- Missing: complex computed fields, definitions
- No encode support (decode-only)

JavaScript (`tools/generate_ts013_codec.py` output)

Generated codecs for TTN/ChirpStack.

- Full decode support
- Partial encode support

- No binary schema (uses generated code)
- Eval-free generated code
- TS013 format compliant
- **v0.3.2 additions:** `downlink_commands` (`encodeCommand/decodeCommand`)

Output JSON Schema (`tools/generate_output_schema.py`)

Validation schemas for decoder output.

- Describes structure of decoded payload data
- JSON Schema draft-07 compliant
- Includes type constraints, ranges, and descriptions
- Enables standard JSON Schema validation of codec output

Schema Validator (`tools/validate_schema.py`)

Schema validation and testing tool (v0.3.2 enhanced).

- Validates schema syntax and structure
- Runs embedded test vectors
- **v0.3.2:** Three-level validation (ERROR/WARNING/INFO)
- Best practice checks for IPSO annotations
- Test coverage recommendations
- JSON output for CI integration

Test Coverage

Test Suite	Python	Go	C
Unit tests			
Test vectors			
Fuzz testing			
Property tests		-	-
Round-trip		Partial	-

Version Compatibility

Schema Version	Python	Go	C	JS
v1 (baseline)				
v2 (extended)				

All implementations MUST support v1 schemas. V2 adds optional features.

Performance Benchmarks

Tested with DL-5TM schema (8 fields, flagged construct, polynomial transform).

Hardware Comparison

Hardware	Year	Python Interpreter	Go Binary Schema
AMD Ryzen 9 7950X3D	2023	81K ops/s (12 μ s)	1.87M ops/s (0.5 μ s)

Hardware	Year	Python Interpreter	Go Binary Schema
Intel i5-2400	2011	17K ops/s (58 μ s)	555K ops/s (1.8 μ s)
Ratio		4.7x	3.4x

Go Implementation (Intel i5-2400)

Implementation	Throughput	Latency
Native Go	1.45M ops/s	690 ns
Binary Schema (pre-parsed)	555K ops/s	1.8 μ s
YAML Schema (pre-parsed)	121K ops/s	8.3 μ s
Binary Parse	393K ops/s	2.5 μ s
YAML Parse	2.2K ops/s	446 μ s

Python Implementation (Intel i5-2400)

Implementation	Throughput	Latency
Native Python	514K ops/s	1.9 μ s
Binary Schema (w/ parse)	28K ops/s	36 μ s
Interpreter (pre-parsed)	17K ops/s	58 μ s
Interpreter (w/ parse)	179 ops/s	5.6 ms

Estimated Cloud Performance

Cloud Instance	Python Interpreter	Go Binary Schema
AWS t3.micro (\$7/mo)	17K ops/s	555K ops/s
AWS t3.small (\$14/mo)	20K ops/s	650K ops/s
AWS c6i.large (\$62/mo)	50K ops/s	1.2M ops/s
AWS c7g.large (Graviton3)	55K ops/s	1.4M ops/s

LoRaWAN Scale Analysis

Devices	Messages/day	t3.micro Python	t3.micro Go
100	14K	<1 sec	trivial
1,000	144K	8 sec	<1 sec
10,000	1.4M	84 sec	3 sec
100,000	14M	14 min	26 sec
1,000,000	144M	2.4 hours	4.3 min

Recommendations

- **<10K devices:** Python on t3.micro (\$7/mo) is sufficient
- **10K-100K devices:** Go on t3.small (\$14/mo) recommended
- **>100K devices:** Go on c6i.medium (\$31/mo) for headroom
- **Latency-sensitive:** Go Binary Schema (<2 μ s on cloud)

Roadmap

Recently Completed (v0.3.2)

Feature	Implementation	Notes
<code>encoding:</code> property	Python, JS	sign_magnitude, bcd, gray
<code>downlink_commands:</code>	Python, JS	Command-based encoding
<code>direction:</code> property	Python, JS	uplink/downlink/bidirectional
<code>use:</code> shorthand	Python	Simplified schema composition
Compact format strings	Python	struct-like syntax
Validation levels	Python	ERROR/WARNING/INFO
Quality scoring	Python	Bronze/Silver/Gold/Platinum

Planned Additions

Feature	Target	Priority
Go encode (full)	Q2	Medium
Go downlink_commands	Q2	Medium
C definitions	Q2	Low
JS binary schema	Q3	Medium
Rust implementation	Q3	High
WASM build	Q4	Medium

Schema Language Enhancements

See FUTURE-FEATURES.md for detailed specifications.

Feature	Value	Status
<code>valid_range</code>	Quality flags, bounds checking	Implemented
<code>resolution</code>	Metadata annotation	Implemented
<code>unece</code>	Standard unit identifiers	Implemented
<code>downlink_commands</code>	Command-based downlinks	Implemented
<code>direction</code>	Schema direction hint	Implemented
Validation levels	ERROR/WARNING/INFO	Implemented
Compact format	Struct-like syntax	Implemented

Not Planned

- Formula expressions (security concern)
- Dynamic schema modification
- Encryption/compression (out of scope)